

Bootcamp: Full Stack Junior

Modulo 3: Backend con PHP y Laravel

Semana 10

Contenido

1

Validaciones y protección de datos en Laravel

2

Mitigación de OWASP Top 10 (SQLi, XSS).

3

Especificación OpenAPI y Swagger UI.

Contenido:

1. Validaciones y protección de datos en Laravel

Para construir aplicaciones robustas en Laravel 12, la seguridad comienza en la capa de entrada. El principio fundamental es **"Never trust user input"** (Nunca confíes en lo que envía el usuario); cada bit de información debe ser tratado como malicioso hasta que se demuestre lo contrario.

1.1. Conceptos de seguridad clave

- **FormRequest:** Es la mejor práctica para desacoplar la lógica de validación del Controlador. Permite que el controlador solo se ejecute si los datos son válidos.
- **Sanitización:** No solo validamos el formato, sino que limpiamos los datos (ej. quitar etiquetas HTML o espacios innecesarios) para evitar ataques XSS o inyecciones.
- **Primera Capa de Defensa:** Validar evita que datos basura lleguen a tu base de datos, previniendo errores de integridad y cuellos de botella.

1.2. Implementación técnica

Creación de un FormRequest

En lugar de validar en el controlador, usa el comando: `php artisan make:request StoreUserRequest`

```
// app/Http/Requests/StoreUserRequest.php

public function rules(): array
{
    return [
        'name'      => 'required|string|max:255',
        'email'     => 'required|email|unique:users,email',
        'password'  => 'required|string|min:8|confirmed', // Requiere password_confirmation
        'username'  => ['required', 'regex:/^[a-zA-Z0-9_]+$/', // Solo letras, números y guion bajo
        'role_id'   => 'required|exists:roles,id', // Verifica que el ID exista en la tabla roles
    ];
}
```

Ilustración

1- FormRequest Laravel 12+ - Fuente: Autor de contenidos

Mensajes de Error Personalizados

Para mejorar la experiencia de usuario (UX), puedes sobrescribir el método `messages()` en

tu Request:

```
public function messages(): array
{
    return [
        'email.unique' => 'Este correo ya está registrado en nuestro sistema.',
        'username.regex' => 'El nombre de usuario solo puede contener letras y números.',
    ];
}
```

Ilustración

2- Custom error messages Laravel 12+ - Fuente: Autor de contenidos

Validaciones en APIs (JSON)

Laravel detecta automáticamente si la petición es una API. Si la validación falla, devuelve un **422 Unprocessable Entity** con un JSON estructurado:

```
{
  "message": "The username format is invalid.",
  "errors": {
    "username": ["El nombre de usuario solo puede contener letras y números."]
  }
}
```

Ilustración 3 - Errores JSON bien estructurados Laravel 12+ - Fuente: Autor de contenidos

1.3. Tipos de Validaciones y Seguridad

Tipo	Ejemplo	Riesgo que previene
Required	email obligatorio	Datos incompletos que rompen la lógica de negocio.
Regex	Formato de teléfono o DNI	Inyección de scripts o formatos de datos inconsistentes.
Exists	role_id válido	Manipulación de IDs (ID Enumeration) y errores de FK.
Unique	Emails duplicados	Corrupción de datos y problemas de autenticación.
Confirmed	password_confirmation	Errores de escritura en datos críticos del usuario.

Tabla 1 - Tipos de validaciones - Fuente: Autor de Contenidos

1.4. Sanitización de Entradas

Laravel 12 incluye middlewares por defecto que limpian los datos (TrimStrings y ConvertEmptyStringsToNull). Sin embargo, para una protección extra contra XSS, asegúrate de usar siempre las llaves de Blade {{ \$variable }} que escapan el contenido

automáticamente.

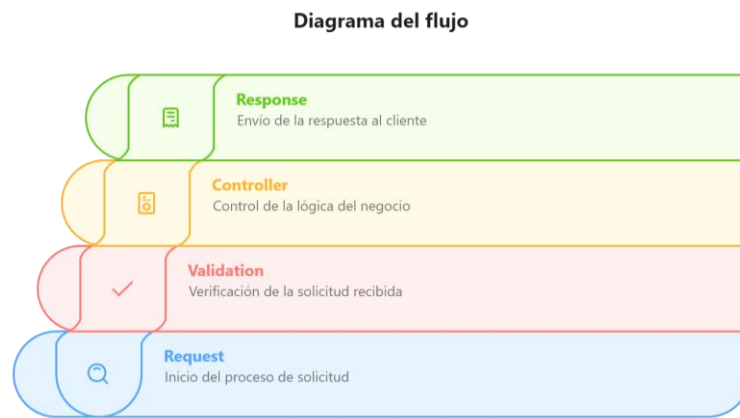


Ilustración 4 - Diagrama de flujo de validaciones en Laravel 12+ - Fuente: Autor de contenidos

2. Mitigación de OWASP Top 10 (SQLi, XSS).

Laravel es uno de los frameworks más seguros "out of the box", pero la seguridad no es un interruptor que se enciende, sino una disciplina de programación. A continuación, detallamos cómo mitigar los dos riesgos más críticos del OWASP Top 10 utilizando las herramientas nativas de Laravel 12.

OWASP top 10:

El **OWASP Top 10** es el estándar de oro para la seguridad web. Laravel mitiga gran parte de estos riesgos mediante su motor de plantillas (Blade) y su capa de abstracción de base de datos (Eloquent), siempre que el desarrollador no intente "saltarse" estas protecciones con consultas crudas o salidas sin procesar.

2.1. SQL Injection (SQLi)

La inyección de SQL ocurre cuando datos proporcionados por el usuario se concatenan directamente en una consulta, permitiendo a un atacante alterar la lógica de la base de datos.

El peligro de las consultas manuales

El uso de `DB::select` o `whereRaw` con variables sin protección es la puerta de entrada al desastre:

```
// ¡EXTREMADAMENTE INSEGURO!
$user = DB::select("SELECT * FROM users WHERE email = '$email'");
```

Ilustración 5 – Consulta vulnerable Laravel 12+ - Fuente: Autor de contenidos

Si \$email contiene ' OR '1'='1', el atacante obtendrá todos los usuarios.

Mitigación en Laravel

Laravel utiliza **Pdo Parameter Binding** para asegurarse de que los datos enviados nunca sean interpretados como comandos SQL.

1. **Eloquent ORM:** Es la forma más segura. Laravel se encarga del binding automáticamente.

```
$user = User::where('email', $email)->first();
```

Ilustración 6 – Consulta con Eloquent Laravel 12+ - Fuente: Autor de contenidos

2. **Query Builder con Binding:** Si necesitas consultas manuales, usa marcadores de posición.

```
$user = DB::table('users')
->whereRaw('email = ?', [$email]) // El '?' es reemplazado de forma segura
->first();
```

Ilustración 7 – Consulta con Query Builder Laravel 12+ - Fuente: Autor de contenidos

2.2. Tabla Comparativa: Seguridad en Consultas

Práctica	Nivel de Riesgo	Protección
Queries Manuales (Concatenación)	Critico	Ninguna. Vulnerable a SQLi directo.
Query Builder (Fluent)	Bajo	Protección automática mediante Bindings.
Eloquent ORM	Minimo	Máxima seguridad y abstracción de datos.

Tabla 2 - Seguridad en consultas - Fuente: Autor de Contenidos

2.3. Cross-Site Scripting (XSS)

El XSS permite a un atacante inyectar scripts (generalmente JavaScript) en las páginas que ven otros usuarios.

Tipos de Ataque

- **Stored XSS:** El script malicioso se guarda en la base de datos (ej. un comentario de blog) y se ejecuta cada vez que alguien visita la página.

- **Reflected XSS:** El script viaja en la URL (ej. un parámetro de búsqueda) y se ejecuta al mostrar el resultado.

Protección en Laravel

1. **Escape Automático en Blade:** Laravel 12 utiliza las llaves dobles `{{ }}` para escapar automáticamente cualquier salida HTML, convirtiendo caracteres peligrosos como `<` en `<`.



```
// Si $name es "<script>alert('hack')</script>"
Hello, {{ $name }}; // Resultado seguro: Hello, &lt;script>&gt;...
```

Ilustración 8 – Seguridad contra XSS Laravel 12+ - Fuente: Autor de contenidos

2. Uso consciente de Raw Output: Solo usa `{!! $variable !!}` cuando estés absolutamente seguro de que el contenido es seguro o ya ha sido sanitizado. Nunca lo uses para entradas directas del usuario.
3. Content Security Policy (CSP): Se recomienda configurar cabeceras CSP para restringir qué scripts pueden ejecutarse en tu aplicación, añadiendo una capa de defensa extra si el filtrado falla.

3. Especificación OpenAPI y Swagger UI.

La documentación técnica no es solo un "manual de usuario", es el contrato que garantiza que el Backend y el Frontend (o terceros) hablen el mismo idioma. En Laravel 12, el estándar para cumplir este contrato es OpenAPI junto con la interfaz visual de Swagger.

3.1. ¿Por qué documentar APIs?

Documentar tu API transforma un "código negro" en un producto accesible y profesional.

- **Facilita el trabajo en equipo:** Los desarrolladores de Frontend no necesitan leer tu código PHP para saber qué enviar.

- **Reduce errores de consumo:** Define claramente qué campos son obligatorios y sus tipos de datos.
- **Testing sin frontend:** Permite probar endpoints, subir archivos y verificar respuestas directamente desde el navegador.
- **Estándar industrial:** OpenAPI es el lenguaje universal para describir servicios RESTful.

3.2. ¿Por qué documentar APIs?

OpenAPI es el **plano arquitectónico** de tu API. Define la estructura de tus servicios mediante archivos YAML o JSON, detallando:

- **Endpoints:** Las rutas disponibles (ej. /api/v1/users).
- **Parámetros:** Datos requeridos en la URL, Query o Body.
- **Esquemas de Datos:** La forma exacta de los objetos que entran y salen.
- **Respuestas:** Qué sucede cuando todo sale bien (\$200\text{ OK}\$) o cuando algo falla (\$422\text{ Validation Error}\$, \$401\text{ Unauthorized}\$).

3.3. Swagger UI en Laravel

Swagger UI es la capa visual que interpreta el archivo OpenAPI y lo convierte en una consola interactiva.

En el ecosistema Laravel, el paquete líder es L5-Swagger (basado en swagger-php). Permite generar la documentación mediante atributos de PHP 8+ directamente en tus controladores:



```
#[OA\Get(
    path: "/api/users",
    summary: "Obtener lista de usuarios",
    responses: [
        new OA\Response(response: 200, description: "Lista de usuarios exitosa"),
        new OA\Response(response: 401, description: "No autorizado")
    ]
)]
public function index() { ... }
```

Ilustración 9 – Documentación con Swagger Laravel 12+ - Fuente: Autor de contenidos

Características principales:

- **Interfaz Interactiva:** Botón "Try it out" para ejecutar peticiones reales.
- **Generación desde Código:** Al actualizar tus atributos en el controlador, la documentación se refresca con un comando (php artisan l5-swagger:generate).
- **Autenticación:** Soporta Bearer Tokens (JWT), Sanctum y OAuth2 de forma nativa.

3.4. Elementos clave de OpenAPI

Elemento	Descripción	Ejemplo
Paths	Los endpoints o rutas de acceso de la API.	/products/{id}
Schemas	Definición de los modelos (objetos) de datos.	User, Product, ErrorResponse
Responses	Listado de códigos de estado HTTP y su significado.	201: Created, 404: Not Found
Security	Métodos de protección para acceder a los recursos.	bearerAuth, apiKey
Parameters	Variables que se pasan por URL o encabezados.	page, per_page, sort

Tabla 3 - Elementos de OpenAPI - Fuente: Autor de Contenido

Recursos de Apoyo



Material de Lectura

[Lectura y ejercicios validaciones estrictas y protección de datos.](#)



Guia

[Guia de mitigación SQLi \(OWASP\).](#)



Material de Lectura

[Documentación con OpenAPI \(Swagger\)](#)